Controle Doméstico Simulado com Modelos Pequenos em SBCs: Sucesso de Tarefas, Latência e Replicação

Hudson Moreira Guimarães dos Santos*, Pablo Cesar Calcina Ccori*,
Laisa Caroline Costa De Biase*, Bruno Cavalcante de Souza Sanches*,
Roseli de Deus Lopes*, Marcelo Knörich Zuffo*
*University of São Paulo (USP), São Paulo, Brazil
Autor correspondente: Hudson Moreira Guimarães dos Santos (hudson.moreira@usp.br)

***Resumo*—Modelos pequenos de linguagem podem ampliar as formulações aceitas por um controlador doméstico, mas sua utilidade depende também da correção das decisões e do custo de resposta. Comparamos regras finitas e modelos Qwen2.5 de 0.5B e 1.5B quantizados em computadores de placa única Labrador. Hardware e inferência são reais; a casa e todas as ações são simuladas. Quarenta cenários reservados em português abrangem pedidos claros, paráfrases, rotinas, ambiguidades e pedidos inválidos, com duas repetições por condição. O sucesso médio por cenário foi 52.5% nas regras, 37.5% no 0.5B e 15.0% no 1.5B; as medianas de latência foram 0.157, 36.848 e 94.663 s. Todas as respostas foram estruturalmente válidas. O 0.5B atendeu mais paráfrases que as regras, mas ambos os modelos tiveram dificuldades com rejeição e esclarecimento. Réplicas independentes em uma, três e seis placas aumentaram as taxas de respostas corretas em lotes fixos, mantendo as contagens de sucesso. O estudo avalia conjuntamente decisões e custo de inferência, distinguindo propostas de intervenções do executor. As conclusões se referem às configurações e ao conjunto construído.**

***Palavras-chave*—casa simulada, modelos pequenos, SBC, sucesso de tarefas, replicação**

I. Introdução

Um controlador doméstico precisa interpretar pedidos, preservar estados de dispositivos não envolvidos e decidir quando pedir informação ou rejeitar uma operação não suportada. Esses requisitos tornam o controle por linguagem natural mais exigente que produzir uma descrição válida de ação. Em um dispositivo com recursos restritos, a correção também precisa justificar o tempo e a memória necessários para obter uma resposta.

Duas linhas de pesquisa informam esse problema: a avaliação de instruções domésticas, inclusive inválidas, e a caracterização de modelos sob restrições de borda. Sua interseção motiva uma pergunta de implantação: como regras e duas configurações de modelos pequenos diferem na execução correta de tarefas domésticas simuladas, no custo de inferência e no atendimento de pedidos independentes por réplicas?

Abordamos essa pergunta com inferência ARM real e um simulador de iluminação de dois cômodos. As contribuições são comparar modelos a uma referência de regras finitas nas mesmas placas Labrador; distinguir saída válida, proposta correta e intervenção do executor; e avaliar lotes fechados instrumentados atendidos por réplicas independentes. Cada placa executa um modelo completo. As réplicas processam pedidos separados, sem colaborar em uma decisão ou particionar um modelo.

II. Trabalhos relacionados

HomeBench [1] avalia instruções em inglês em uma casa virtual, abrangendo pedidos válidos, inválidos e mistos para um ou vários dispositivos. Sua métrica de sucesso exige a execução correta de todas as operações solicitadas; F1 descreve adicionalmente o desempenho por operação. Compara modelos e estratégias de adaptação, em vez de uma implantação equivalente em Labrador. Nosso conjunto menor, em português, focaliza esclarecimento de cômodo ausente e preservação de estados não envolvidos, registrando ações propostas, bloqueadas e aplicadas separadamente. O conjunto foi construído de forma independente, e seus percentuais não são comparados aos de HomeBench.

Lu et al. [2] caracterizam mais de 60 modelos pequenos com benchmarks amplos de capacidade e experimentos de custo de implantação. A capacidade é avaliada em servidor, enquanto o custo inclui smartphones e hardware Jetson, separando processamento de prompt e geração e examinando quantização. Nossa comparação mede decisões de tarefas e latência HTTP na mesma plataforma Labrador, usando somente CPU. Trata de uma tarefa restrita de controle, em vez de rankings gerais, relacionando custo por pedido à aceitabilidade da ação resultante.

Qin et al. [3] examinam como restrições de recursos afetam escolhas de modelos personalizados, incluindo métodos de aprendizagem, volume de dados, compressão, dificuldade da tarefa e tempo disponível para aprender. O foco em customização difere de nossas configurações pré-treinadas fixas: realizamos inferência sem treinamento ou personalização. Em conjunto, esses estudos motivam avaliar requisitos da tarefa e custo de implantação. Acrescentamos uma comparação rastreável entre regras, modelos pequenos e replicação independente para controle simulado em português. A comparação é meto-

dológica; diferenças de tarefas, modelos, idioma e hardware impedem rankings numéricos diretos entre os estudos.

III. Método experimental

A. Simulador e política de execução

Um coordenador Python cria um estado novo por pedido, consulta um serviço HTTP residente, valida o JSON, aplica mudanças permitidas e avalia o resultado separadamente. Nenhum dispositivo doméstico está conectado. Os cômodos são identificados pelos valores técnicos `sala` e `quarto`; as luzes assumem `on`/`off`. O contrato admite `set_light`, `set_lights`, `ask_clarification` e `no_action`; planos de dois passos são atômicos, de modo que uma falha de permissão bloqueia o plano inteiro.

O executor vê apenas pedido, estado visível e permissões, sem gabaritos ou categorias dos cenários. Uma política conservadora bloqueia propostas de iluminação se nenhum nome conhecido de cômodo aparece como palavra inteira no pedido e não há localização válida. Cômodos desconhecidos e valores não autorizados também são bloqueados. Essa política verifica permissão, não correção da tarefa: uma proposta permitida ainda pode mudar a luz errada ou interpretar incorretamente uma instrução inválida. Propostas, intervenções e resultados são registrados separadamente.

B. Hardware e configurações

Seis placas Labrador possuem quatro núcleos ARM64, Debian 12 e cerca de 1.89 GiB de RAM visível ao sistema por placa. Os modelos usam `llama-server` [4] build 9584/`e25a32e98`, execução em CPU, quatro threads de computação e quatro de lote, contexto de 512 tokens e um slot por serviço. Os dois serviços de modelo permanecem residentes em cada placa, com no máximo uma inferência ativa entre eles. Qwen2.5-0.5B-Instruct e Qwen2.5-1.5B-Instruct [5], [6] usam arquivos GGUF Q5_K_M cujos hashes coincidem com os oficiais da Qwen [7], [8]. Hashes de runtime e bibliotecas foram conferidos entre placas; a proveniência completa está no artefato.

As regras executam em serviço HTTP Python residente e sem estado nas mesmas placas. A gramática finita reconhece verbos fixos em português para ligar ou desligar luzes, cômodos explícitos, conjunções simples, comandos com localização conhecida e expressões de cortesia nas extremidades do pedido. Outras formas retornam `no_action`. Trata-se de uma referência limitada de engenharia, não de um sistema ótimo de linguagem baseado em regras.

C. Conjunto de pedidos e critérios de sucesso

Oito casos de desenvolvimento precederam o congelamento; essas campanhas permanecem separadas da avaliação reservada. Cenários, estados e passos esperados, prompts, esquema, política de execução, configurações, repetições e scripts de medição foram congelados antes do acesso às saídas reservadas. Há oito cenários em cada categoria: pedidos claros, paráfrases, rotinas explícitas, ambiguidades e pedidos inválidos. Duas repetições produzem 80 pedidos por condição e 240 no total. Todas as condições executam em todos os hosts; as repetições de um cenário usam placas diferentes, com pedidos sequenciais por placa.

As requisições dos modelos combinam instruções gerais em inglês e texto do usuário em português. A temperatura é zero, as sementes são 801/802, a saída é limitada a 128 tokens e o timeout é 180 s. O cache de prompt é desativado. As repetições avaliam estabilidade operacional, com diversidade limitada de decisões. Após timeout, o coordenador aguarda ociosidade dos dois serviços antes de outro pedido.

Sucesso exige conclusão normal, JSON válido pelo contrato, proposta correta, estado final esperado, preservação de estados não envolvidos e ausência de passos extras ou proibidos. Uma ação de luz e um plano equivalente de um passo são aceitos igualmente. Rotinas exigem exatamente os dois passos esperados. Pedidos ambíguos exigem uma pergunta informativa sobre o cômodo ausente e luzes inalteradas; inválidos exigem `no_action` e luzes inalteradas. Portanto, atingir o estado-alvo ou bloquear uma proposta incorreta é insuficiente isoladamente.

Perguntas de esclarecimento são verificadas com padrões conservadores de pergunta completa; mencionar um cômodo é insuficiente. Outras perguntas recebem revisão caso a caso, com identidade e justificativa registradas. Quatro julgamentos assistidos permanecem pendentes de validação humana e não recebem crédito de sucesso. As duas repetições são agrupadas por cenário e cada cenário recebe peso igual. Contagens de pedidos não são tratadas como cenários independentes para inferência populacional.

D. Latência, memória e replicação

O relógio monotônico do coordenador mede do despacho HTTP até a leitura e decodificação, incluindo comunicação e inferência, mas excluindo carregamento e instantâneos SSH. Mediana e p95 por posto mais próximo descrevem pedidos individuais. Logs do runtime fornecem tokens de entrada e saída. Instantâneos RSS descrevem memória do processo antes e depois do pedido; VmHWM é o máximo desde o início do processo. Nenhum representa pico por pedido ou memória física exclusiva do modelo, pois serviços permanecem residentes e mapeamentos podem ser compartilhados.

A replicação usa uma, três ou seis placas ativas, com um pedido por placa em cada onda e dois lotes de seis pedidos por tamanho e condição: 108 pedidos no total. Cada lote contém um pedido claro, uma paráfrase, duas rotinas, uma ambiguidade e um inválido, cada um com estado novo e sem histórico de conversa. A janela vai do primeiro despacho à última resposta HTTP, incluindo intervalos instrumentados entre ondas. Respostas/min e respostas corretas às tarefas/min são separadas. Simulação e avaliação ocorrem após o recebimento, portanto a segunda taxa não mede tempo até mudar o estado. Esse conjunto fechado avalia atendimento em lotes, não saturação ou capacidade máxima.

Tabela I
SUCESSO, LATÊNCIA E COMANDOS DE ILUMINAÇÃO.

Condição	Sucesso	Mediana	p95	Indevidos	Bloqueados
Regras	42/80	0.157	0.643	0	0
Qwen 0.5B	30/80	36.848	45.292	36	16
Qwen 1.5B	12/80	94.663	159.117	76	22

Tabela II
SUCESSO POR CATEGORIA DE PEDIDO.

Categoria	Regras	0.5B	1.5B
Instruções claras	16/16	8/16	0/16
Paráfrases	0/16	14/16	4/16
Rotinas	8/16	8/16	6/16
Ambiguidade	2/16	0/16*	0/16
Pedidos inválidos	16/16	0/16	2/16

IV. RESULTADOS

A. Sucesso de tarefas e custo de inferência

A Tabela I resume 80 pedidos de 40 cenários por condição. Latências estão em segundos; as duas últimas colunas contam comandos individuais de luz, inclusive dentro de planos, em vez de respostas. Todas as 240 respostas terminaram normalmente e satisfizeram o contrato JSON. Ainda assim, o sucesso diferiu substancialmente: regras concluíram 42/80 tentativas, o 0.5B concluiu 30/80 e o 1.5B concluiu 12/80.

A comparação por categoria na Tabela II identifica onde as abordagens diferiram. Cada célula abrange oito cenários com duas repetições, ou 16 tentativas; não são 16 cenários independentes. Regras acertaram todos os pedidos claros e inválidos, mas nenhuma paráfrase. O 0.5B acertou 14/16 paráfrases, mostrando maior cobertura dessas formulações, mas não rejeitou corretamente nenhum pedido inválido. Nenhum modelo tratou ambiguidades com sucesso pelos critérios registrados.

O asterisco indica quatro julgamentos de esclarecimento do 0.5B que aguardam validação humana. Permanecem provisoriamente malsucedidos; todos os sucessos creditados são automáticos. Aceitar os quatro produziria 34/80 (42.5%) no 0.5B, ainda abaixo das regras com 42/80 (52.5%); esse limite de sensibilidade não resolve os julgamentos.

O 0.5B também teve menor latência que o 1.5B: medianas de 36.848 e 94.663 s, contra 0.157 s nas regras. RSS mediano após o pedido foi 571.6 e 1183.7 MiB nos modelos. Ambos usaram 124–138 tokens de entrada; as saídas variaram de 16–40 e 12–75 tokens, respectivamente. As contagens de cache de prompt foram zero. Essas medidas caracterizam configurações completas, sem isolar o efeito do número de parâmetros.

B. Falhas de decisão e casos representativos

A Tabela III agrupa verificações existentes sem alterar resultados. Cada célula apresenta tentativas reprovadas/observadas. Passos e estado-alvo se aplicam a 48 tentativas claras, paráfrases e rotinas; preservação se aplica a 64 tentativas, excluindo rotinas. Linhas se sobrepõem e não podem ser somadas. Proibições são verificadas em todas as 80 tentativas, embora apenas 32 ambíguas/inválidas tenham proibições não vazias. Esclarecimento e rejeição avaliam propostas desses tipos; ausência de propostas de esclarecimento não demonstra essa capacidade.

Tabela III
REQUISITOS REPROVADOS NAS TENTATIVAS OBSERVADAS.

Requisito	Regras	0.5B	1.5B
Passos incorretos	24/48	18/48	38/48
Estado-alvo incorreto	24/48	18/48	22/48
Falha de preservação	0/64	8/64	14/64
`no_action` indevido	38/54	4/4	20/22
Proposta bloqueada	0/80	14/80	12/80
Passo proibido	0/80	26/80	24/80
Esclarecimento inadequado	0/2	12/12*	–/0

Regras preservaram estados não envolvidos, mas deixaram de agir em pedidos válidos fora da gramática. Os modelos apresentaram outro perfil: conjuntos de passos incorretos, propostas proibidas e mudanças em luzes não envolvidas. O 1.5B reprovou igualdade de passos em 38/48 tentativas aplicáveis, mas estado-alvo em 22/48, mostrando por que a luz solicitada isoladamente não estabelece conclusão da tarefa. Por comando, o 0.5B propôs 36 mudanças indevidas, com 16 bloqueadas e 20 simuladas; o 1.5B propôs 76, com 22 bloqueadas e 54 simuladas. Essas contagens diferem das tentativas bloqueadas, pois uma resposta pode conter vários comandos e o plano é bloqueado atomicamente.

A Tabela IV apresenta quatro casos registrados. Pedidos e textos de resposta mantêm o português original; chaves e valores técnicos são preservados, e a descrição de ações abrevia o JSON registrado. A versão inglesa apresenta traduções explicitamente identificadas. Enunciados originais, saídas completas, estados e o mapeamento dos rótulos para diretórios estão no suplemento. A contagem de esclarecimentos inadequados inclui os quatro julgamentos provisórios já identificados.

O Caso 1 mostra uma paráfrase válida rejeitada pelas regras. O Caso 2 separa a suposição incorreta de cômodo de sua execução bloqueada. O Caso 3 ilustra uma resposta no campo de pergunta que, provisoriamente, não solicita a informação ausente. O Caso 4 atinge o estado solicitado, mas altera o outro cômodo, reprovando preservação e passos exatos. Esses casos explicam as distinções de avaliação sem atribuir causas internas não medidas aos modelos.

C. Réplicas independentes

A Tabela V e a Fig. 1 descrevem atendimento em lotes. Cada linha reúne 12 pedidos para sucesso e latência; taxas são médias de dois valores por lote. Latências estão em segundos e a taxa expressa respostas corretas às tarefas/min na janela HTTP. No 0.5B, a taxa média subiu de 0.871 para 2.883 e 6.393/min com uma, três e seis placas; no 1.5B, de 0.085 para 0.289 e 0.661/min. Cada lote de seis pedidos manteve três sucessos no 0.5B, um no 1.5B e cinco nas regras. A

Tabela IV
FALHAS DE DECISÃO REPRESENTATIVAS.

Caso	Pedido e comportamento esperado	Resposta observada	Requisito não atendido
1. **Paráfrase não reconhecida**	"Por favor, deixe a luz da sala ligada." Ligar `sala`; preservar `quarto=on`.	Regras: `no_action`; motivo "Pedido fora das regras explícitas." A luz solicitada permanece apagada.	Passos e estado-alvo corretos; rejeição é indevida para esse pedido válido.
2. **Suposição de cômodo**	"Ligue a luz, por favor." Sem localização conhecida; perguntar o cômodo e preservar as luzes.	0.5B: `set_light`, `room=sala`, `value=on`. Executor bloqueia a proposta; estado inalterado.	Ação correta, permissão e ausência de passos proibidos. Bloqueio impede aplicação, mas não corrige a proposta.
3. **Esclarecimento não informativo**	"Quero a luz acesa." Sem localização conhecida; perguntar o cômodo e preservar as luzes.	0.5B: `ask_clarification`, `missing_field=room`; `question`: "Quero a luz acender." Sem mudança de estado.	Pergunta informativa, provisoriamente: o julgamento assistido de falha aguarda validação humana.
4. **Mudança não solicitada**	"Ligue a luz na sala." Ligar `sala`; preservar `quarto=on`.	1.5B: `set_lights` com `(sala,on)` e `(quarto,off)`. Ambos os comandos permitidos e simulados.	Passos exatos e preservação. A luz-alvo está correta, mas o quarto também é alterado.

Tabela V
RESPOSTAS ÀS TAREFAS COM RÉPLICAS INDEPENDENTES.

Condição	Placas	Sucesso	Taxa	Mediana	p95
Regras	1	10/12	42.753	0.006	0.211
Regras	3	10/12	235.196	0.006	0.008
Regras	6	10/12	30555.536	0.006	0.011
Qwen 0.5B	1	6/12	0.871	24.586	37.903
Qwen 0.5B	3	6/12	2.883	23.993	40.917
Qwen 0.5B	6	6/12	6.393	23.990	28.772
Qwen 1.5B	1	2/12	0.085	84.187	131.203
Qwen 1.5B	3	2/12	0.289	78.606	139.278
Qwen 1.5B	6	2/12	0.661	80.019	93.518

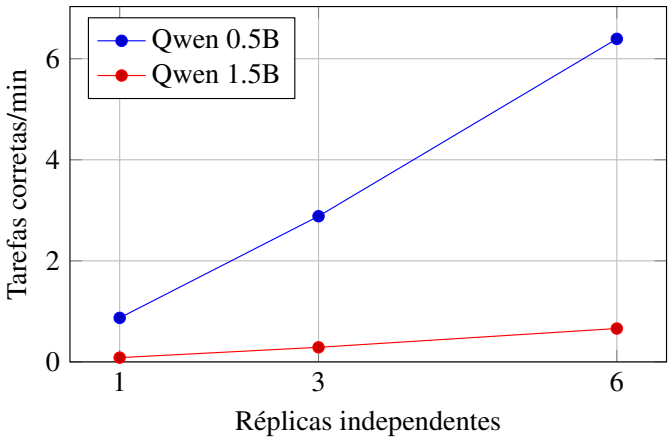


Figura 1. Taxas de respostas corretas em lotes fechados.

taxa elevada das regras com seis réplicas reflete uma janela de cerca de 9–11 ms e a remoção dos intervalos instrumentados entre ondas, não capacidade sustentada medida. Janelas e taxas de resposta estão disponíveis separadamente no artefato. Nos lotes testados, a replicação aumentou atendimento concorrente, mantendo as contagens de sucesso. Em conjunto, os resultados distinguem três aspectos: todas as configurações satisfizeram o contrato, categorias expuseram diferentes falhas de decisão e mais réplicas aumentaram taxas de resposta sem corrigir essas falhas.

V. DISCUSSÃO

A. *Escolha da abordagem para a tarefa*

Regras tiveram menor custo e maior sucesso agregado neste conjunto, mas sua falha completa em paráfrases limita a interpretação desse ranking. O 0.5B cobriu mais desses pedidos, falhando em rejeição de inválidos e em várias verificações de preservação. A escolha depende, portanto, dos pedidos que precisam ser suportados e dos erros aceitáveis, junto aos limites de latência e memória. A evidência atual não estabelece um controlador combinado confiável; esse projeto exigiria validação independente.

A distinção entre sintaxe e execução complementa a ênfase de HomeBench na execução completa das instruções [1]. Nossos casos de cômodo ausente e preservação mostram ainda como uma proposta bloqueada ou uma luz-alvo correta podem ocultar uma decisão malsucedida. Seu valor é diagnóstico, sem demonstrar que nosso conjunto menor seja mais abrangente que HomeBench.

A comparação de custo estende a perspectiva de implantação de Lu et al. [2] a esta tarefa em Labrador: medir a memória de um modelo ou gerar JSON válido é insuficiente para escolhê-lo para controle. A configuração 1.5B mais lenta também concluiu menos tarefas. Essa observação se refere ao prompt, quantização e modelos testados; não identifica uma causa ligada ao tamanho. Qin et al. [3] também situam a seleção nas restrições de tarefa e recursos, mas seu contexto de customização não explica nosso ranking. Testar um modelo adaptado exigiria novos dados de desenvolvimento e protocolo congelado separado.

Replicação trata de outra decisão de implantação. Cópias paralelas aumentaram taxas de respostas corretas para pedidos

independentes, com medianas individuais semelhantes entre números de réplicas e contagens de sucesso inalteradas por lote. Não melhoraram a interpretação de um pedido. Avaliar atendimento sustentado exigiria um processo de chegadas, medidas de fila e execuções mais longas que esses lotes instrumentados.

B. Validade e alcance

As 40 combinações elaboradas de enunciado e estado abrangem duas luzes em português, não casas independentes ou uma população de usuários. Várias expressam a mesma ação. Luzes solicitadas começam no estado oposto, sem testar pedidos idempotentes ou outras distribuições de estado; sucesso em rotinas isoladamente não distingue compreensão de inversão de estado. A gramática finita é uma referência limitada, e um prompt, esquema e quantização por modelo não isolam efeitos de arquitetura ou tamanho.

Intercalar hosts reduz confundimento sem eliminar variações térmicas, de memória ou rede. Serviços residentes e mapeamentos compartilhados limitam interpretação da memória; caches de páginas não foram esvaziados e energia e picos por pedido não foram medidos. Poucos lotes fechados, com custos de instrumentação, limitam a generalização das taxas. Quatro julgamentos assistidos permanecem provisórios. A simulação não demonstra segurança ou usabilidade residencial real. Esses limites definem o alcance da comparação de configurações sem alterar os resultados observados.

VI. Conclusão

Nas tarefas domésticas simuladas testadas, regras alcançaram maior sucesso agregado com menor custo de resposta, enquanto o Qwen 0.5B atendeu mais paráfrases que a gramática finita e superou a configuração 1.5B em sucesso e latência. Saída estruturada válida não garantiu decisões corretas: rejeição, esclarecimento e preservação exigiram avaliação separada. Réplicas independentes aumentaram taxas de respostas corretas em lotes fechados, mantendo as contagens de sucesso.

Selecionar um controlador exige, portanto, compatibilizar pedidos suportados e critérios de decisão com o orçamento de inferência. O limite principal é o conjunto pequeno e elaborado para dois cômodos. A próxima etapa concreta é uma nova avaliação revisada independentemente, com formulações de usuários, comandos idempotentes e estados iniciais variados, preservando a distinção entre propostas, intervenção do executor e sucesso da tarefa.

Disponibilidade de evidências e uso de IA

Protocolo congelado, respostas brutas, todas as 240 tentativas de qualidade e 108 de replicação, falhas, registro de revisão e scripts offline estão em snapshot imutável [9]. A análise complementar de falhas preserva exemplos originais e caminhos das fontes. O mapeamento de rótulos e as notas de tradução acompanham esta revisão editorial no repositório. A reconstrução offline confere estado, permissões e resultados sem nova inferência; replicação independente em hardware exige também runtime e modelos registrados.

Não houve financiamento externo. OpenAI Codex auxiliou código, análise, figuras, conferência de evidências e literatura, redação e revisão do manuscrito. Seus quatro julgamentos provisórios de esclarecimento não são revisão humana. Os autores permanecem responsáveis pela verificação e pelo texto final, conforme orientação IEEE sobre uso de IA [10].

Referências

[1] S. Li, Y. Guo, J. Yao, Z. Liu, and H. Wang, "HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 12 230–12 250. [Online]. Available: https://aclanthology.org/2025.acl-long.597/

[2] Z. Lu, X. Li, D. Cai, R. Yi, F. Liu, W. Liu, J. Luan, X. Zhang, N. D. Lane, and M. Xu, "Demystifying small language models for edge deployment," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 14 747–14 764. [Online]. Available: https://aclanthology.org/2025.acl-long.718/

[3] R. Qin, D. Liu, C. Xu, Z. Yan, Z. Tan, Z. Jia, A. Nassereldine, J. Li, M. Jiang, A. Abbasi, J. Xiong, and Y. Shi, "Empirical guidelines for deploying LLMs onto resource-constrained edge devices," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 5, 2025, art. 70. [Online]. Available: https://doi.org/10.1145/3736721

[4] ggml-org, "llama.cpp server documentation," accessed: 2026-10-20; deployed source revision e25a32e98c15a1118a976685af0f4d2c1f380079. [Online]. Available: https://github.com/ggml-org/llama.cpp/blob/e25a32e98c15a1118a976685af0f4d2c1f380079/tools/server/README.md

[5] Qwen, "Qwen2.5-0.5B-Instruct: Official model card," accessed: 2026-10-05. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

[6] ——, "Qwen2.5-1.5B-Instruct: Official model card," accessed: 2026-10-05. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[7] ——, "Qwen2.5-0.5B-Instruct Q5_K_M: Official GGUF file and SHA-256," accessed: 2026-10-06. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF/blob/main/qwen2.5-0.5b-instruct-q5_k_m.gguf

[8] ——, "Qwen2.5-1.5B-Instruct Q5_K_M: Official GGUF file and SHA-256," accessed: 2026-10-06. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF/blob/main/qwen2.5-1.5b-instruct-q5_k_m.gguf

[9] Labrador Distributed LLM Benchmarks contributors, "Simulated smart-home control: Frozen protocol, raw attempts, and reconstruction scripts," GitHub repository (snapshot a834b82), 2026, evidence snapshot a834b82, accessed 2026-10-20.

[10] IEEE, "Guidelines for artificial intelligence (AI)-generated text," 2024, accessed: 2026-10-20. [Online]. Available: https://open.ieee.org/author-guidelines-for-artificial-intelligence-ai-generated-text/